

B 2.4.3 Bubble Sort and Selection Sort in Python



2.4.3

Master the basics of sorting. This section explores two fundamental sorting algorithms: bubble sort and selection sort. Learn how to construct, trace, and implement these algorithms in Java and Python. Analyse their efficiency with Big O notation and understand how their performance compares in different scenarios.

Introduction

Sorting is a fundamental operation in computer science that arranges data in a specific order. In this chapter, we'll explore two classic sorting algorithms: bubble sort and selection sort. We'll learn how they work, how to implement them in Java, and how to analyse their efficiency.

Bubble Sort

Bubble sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. The algorithm, a comparison sort, is named for how smaller or larger elements "bubble" to the top of the list.

Implementation

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap in Python
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

Tracing the Algorithm

Let's trace the execution of the bubbleSort method for the array :

```
arr = [5,1,4,2,8]
```

Pass 1:

- > (5 and 1 are swapped)
- > (5 and 4 are swapped)
- > (5 and 2 are swapped)
- > (5 and 8 are not swapped)

Pass 2:

- > (1 and 4 are not swapped)
- > (4 and 2 are swapped)
- > (4 and 5 are not swapped)

Pass 3:

- > (1 and 2 are not swapped)
- > (2 and 4 are not swapped)

Pass 4:

- > (1 and 2 are not swapped)

The array is now sorted.

Efficiency Analysis

Time Complexity:

- Worst and Average Case: $O(n^2)$
- Best Case: $O(n)$ (if the array is already sorted)

Space Complexity:

- $O(1)$

Selection Sort

Selection sort is another simple sorting algorithm. It works by repeatedly finding the minimum element from the unsorted part of the array and putting it at the beginning.

Implementation

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n - 1):  
        min_index = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_index]:  
                min_index = j  
        # Swap in Python  
        arr[i], arr[min_index] = arr[min_index], arr[i]
```

Tracing the algorithm

Let's trace the execution of the selectionSort method for the array :

```
arr = [5,1,4,2,8]
```

Iteration 1:

Minimum element: 1 (at index 1)

Swap: -> [1,5,4,2,8]

Iteration 2:

Minimum element: 2 (at index 3)

Swap: -> [1,2,4,5,8]

Iteration 3:

Minimum element: 4 (at index 2)

Swap: -> (no change)

Iteration 4:

Minimum element: 5 (at index 3)

Swap: -> (no change)

The array is now sorted.

Efficiency Analysis

- Time Complexity: $O(n^2)$ in all cases
- Space Complexity: $O(1)$

Comparing Bubble Sort and Selection Sort

Both have the same average and worst-case time complexity ($O(n^2)$).

Bubble sort can be more efficient for nearly sorted data ($O(n)$ in the best case).

Selection sort generally performs fewer swaps.

Key Questions

Hinge Question: Why does bubble sort have a best-case time complexity of $O(n)$ while selection sort always has $O(n^2)$?

Answer: Bubble sort can detect if the array is already sorted and terminate early, leading to $O(n)$ in the best case. Selection sort always performs the nested iterations to find the minimum element, even if the array is already sorted.

Hinge Question: Explain why selection sort might be preferable to bubble sort when swaps are expensive.

Answer: Selection sort performs significantly fewer swaps than bubble sort. In bubble sort, multiple swaps might occur in each pass through the array. In contrast, selection sort performs only one swap per iteration of the outer loop.

Hinge Question: Imagine you need to sort a large dataset and have limited memory available. Would bubble sort or selection sort be a better choice? Why?

Answer: Both bubble and selection sort are in-place sorting algorithms, meaning they have a space complexity of $O(1)$. This makes them suitable for situations with limited memory because they don't require significant extra space to sort.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.
